

Warm up

Why is the following code incorrect? Any other possible memory issues?

```
struct Mystery {  
    int* a;  
    int* b;  
};  
  
void free_mystery(struct Mystery* m) {  
    free(m);  
    free(m->a);  
    free(m->b);  
}
```

Warm up

Why is the following code incorrect? Any other possible memory issues?

```
struct Mystery {  
    int* a;  
    int* b;  
};  
  
void free_mystery(struct Mystery* m) {  
    free(m);  
    free(m->a);  
    free(m->b);  
}
```

- If either `m->a` or `m->b` is not dynamically allocated (e.g., if they point to stack memory or already deallocated) → invalid `free`
 - Unfortunately cannot be independently verified by `free_mystery`
- If `m->a` and `m->b` point to the same address → double free
- `free(m)` is called before `m->a` and `m->b` are free'd → use-after-free (dangling pointer)

Warm up

(Semi-)corrected version of `free_mystery`:

```
void free_mystery(struct Mystery* m) {  
    free(m->a);  
    free(m->b);  
    free(m);      /* free the struct itself last */  
}
```

- If `free_mystery` called with non-heap allocated memory or the pointers point to the same address, the behaviour is still undefined
 - This should be well-documented in the function contract (e.g., in comments or documentation), and the caller is responsible for ensuring that the preconditions are met
 - If caller violates the contract, then garbage in, garbage out (GIGO)

Precept 10: Abstract Data Types

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 3, 2026

Overview

- 1 Introduction to abstract data types and stacks
- 2 An implementation of the stack abstract data type
- 3 Summary

What is an abstract data type?

- An abstract data type (ADT) is a data structure that encapsulates data and operations on that data, while hiding the implementation details from the user
 - In Java, an ADT can be implemented as a class with private fields and public methods
 - But, C is not an object-oriented language → use structs instead
- ADTs provide an *interface* for users to interact with the data structure → modularity
- Examples of ADTs:
 - linked list
 - queue
 - binary search tree
 - hash table (Assignment 3)
 - stack (today)

Implementing a stack

A stack is a data structure that follows the last-in-first-out (LIFO) principle. It supports two main operations:

- push: add an element to the top of the stack
- pop: remove and return the element at the top of the stack

Overview

- 1 Introduction to abstract data types and stacks
- 2 An implementation of the stack abstract data type
- 3 Summary

Stack interface

We can define the stack interface as follows:

```
struct Stack {  
    double *pdArray;      /* array where items are stored */  
    size_t uTop;          /* index of next empty slot */  
    size_t uPhysLength;   /* number of elements in array */  
};
```

What operations should we support?

- `struct Stack *Stack_new(void);`
- `void Stack_free(struct Stack *psStack);`
- `int Stack_push(struct Stack *psStack, double dItem);`
- `double Stack_pop(struct Stack *psStack);`
- `int Stack_isEmpty(struct Stack *psStack);`

Note: functions take a pointer to a `struct Stack` as an argument, since we want to modify the stack in-place

Where should we put the struct Stack definition?

- In header file: then the client code can access the fields directly through `#include` → breaks encapsulation
- Instead: provide an opaque type to the client code, and define the struct in the implementation file

```
/* stack.h */
struct Stack; /* forward declaration */

/* stack.c */
#include "stack.h"
struct Stack {
    double *pdArray; /* array where items are stored */
    size_t uTop;      /* index of next empty slot */
    size_t uPhysLength; /* number of elements in array */
};
```

struct Stack definition in the implementation file

- Type declaration in header file informs compiler that struct Stack is a valid type
- But: at compile time, compiler sees an incomplete type in client code
 - Client code cannot access the fields of struct Stack directly
 - Compiler does not know the size of struct Stack → cannot allocate memory for it
 - ◇ Cannot declare variable of type struct Stack (e.g., struct Stack s;)
 - ◇ But can declare pointer to struct Stack (e.g., struct Stack *psStack;)
 - Makes the data opaque to the client code → encapsulation!

Using typedef

- typedef allows us to create an alias for an existing type, which can make our code cleaner and easier to read
 - Does not create a new type

```
/* stack.h */  
typedef struct Stack *Stack_T;
```

- Stack_T is now an alias for struct Stack *
- Client code can use Stack_T instead of struct Stack * for better readability
 - Creates an opaque pointer: client code still cannot access the fields of struct Stack directly

Using a linked list instead of an array

- We can also implement a stack using a linked list instead of an array
- The interface remains the same, but the implementation changes
- This is the power of abstract data types: we can change the implementation without affecting the client code, as long as we maintain the same interface
- Trade-offs?
 - Array: faster access to elements, but may require resizing and can waste space if the stack is not full
 - Linked list: more flexible with memory usage, but slower access to elements and more overhead due to storing pointers
- What functions are affected in our stack implementation?
 - `free` is slow for linked list implementation, since we need to free each node individually
 - `push` may be slower for array implementation if we need to resize array
 - Are `new`, `pop` and `isEmpty` affected by the choice of implementation?
 - ◇ Not really, since we keep track of the top of the stack

Food for thought

- In what scenarios would you prefer one implementation?
 - Array: when we have a good estimate of the maximum size of the stack and want faster access to elements
 - Linked list: when maximum size is unbounded
 - In practice: arrays have better cache performance, so they are often preferred for the common case
- What changes would you make to the interface to implement a queue (first-in, first-out) instead?
 - Need to add functions to add elements to the back of the queue (enqueue) and remove elements from the front of the queue (dequeue)
 - Keep track of both the front and back of the queue for efficient access
- How would you modify the stack implementation so that it can support any generic type (not just doubles)?
 - Next precept!

Overview

- 1 Introduction to abstract data types and stacks
- 2 An implementation of the stack abstract data type
- 3 Summary

Summary

- Abstract data types
- Implementing a stack ADT
- Dates to remember:
 - Midterm: Wed, 3/4, 10:40–11:30am (tomorrow!)
 - Assignment 3: due Tue, 3/24 at 9pm
 - Assignment 3 Quiz: Wed, 3/25 in lecture
- Acknowledgements
 - The stack implementations in these slides are adapted from the precept materials, written by Prof. Bob Dondero
 - Can be accessed on armlab at `~cos217/PreceptDemos/precept10` or <https://www.cs.princeton.edu/courses/archive/spring26/cos217/precepts/w05p2/>